



Agile methodologies and Scrum



Before Agile



Life before agile – waterfall

Waterfall is the process of developing a software product by following a **linear development life cycle**.



Process is linear and does not account for corrections!



Life before agile – waterfall

Problems:

- Lots of effort to analyse and design the whole product
- No space for changes
- Overly regulated and micromanaged
- Lots of friction between the company and the client once – **inevitably** – changes are requested
- **Customer is an enemy**



Life before agile – waterfall

Is Waterfall the devil?

- Still good for specific industries (i.e.: military and defense, healthcare and over highly regulated / compliance heavy industries)
- Still good for specific projects, especially on standardized applications!



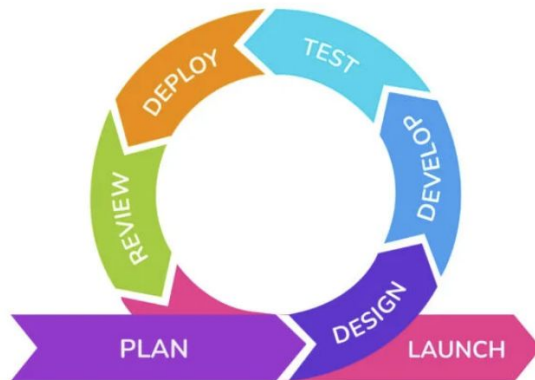
Agile



Agile – why?

Agile

- subverts the old approaches by offering something that does not antagonize the client, but transforms it into an ally – often “tricking” it into working for/with us!
- Iterate, increment and evolve!
- empowers development teams enabling them to be self-organized and cross functional





Agile – how?

Smart people realized the problems with waterfall approaches decades ago, but Agile became popularized on in early 2000s, with the publication of the **Agile Manifesto and its principles**

<https://agilemanifesto.org>

<https://agilemanifesto.org/principles.html>



Manifesto for Agile Software Development

We are uncovering better ways of developing software by doing it and helping others do it.

Through this work we have come to value:

Individuals and interactions over processes and tools

Working software over comprehensive documentation

Customer collaboration over contract negotiation

Responding to change over following a plan

That is, while there is value in the items on the right, we value the items on the left more.

Kent Beck	James Grenning	Robert C. Martin
Mike Beedle	Jim Highsmith	Steve Mellor
Arie van Bennekum	Andrew Hunt	Ken Schwaber
Alistair Cockburn	Ron Jeffries	Jeff Sutherland
Ward Cunningham	Jon Kern	Dave Thomas
Martin Fowler	Brian Marick	

© 2001, the above authors
this declaration may be freely copied in any form,
but only in its entirety through this notice

[Twelve Principles of Agile Software](#)

[View Signatories](#)



Principles behind the Agile Manifesto

We follow these principles:

Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.

Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.

Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale.

Business people and developers must work together daily throughout the project.

Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done.

The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.

Working software is the primary measure of progress.

Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.

Continuous attention to technical excellence and good design enhances agility.

Simplicity--the art of maximizing the amount of work not done--is essential.

The best architectures, requirements, and designs emerge from self-organizing teams.

At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.



Honorable mentions



Lean

From Toyota ~a long long time ago,

- The “classic”
- Define value
- Customer “pulls” change
- Create flows and cut waste



XP

From Kent Beck ~90s,

- Focus on technical excellence (coding)
- Coding as a collaborative activity
- TDD, CI/CD, etc...



Kanban

Taiichi Ohno ~a long long time ago,

- Tasks visualization
- Flow and Kaizen (continuous improvement)
- WIP (Work In Progress)
- Perfect for handling maintenance tasks and projects
- Simple and effective



Scrum



Scrum – why?

- Scrum is the most popular Agile framework adopted in modern software development companies
- Scrum has a good compromise of number of processes, complexity, flexibility, output and number of people needed to manage it



Scrum – how?

Scrum is composed of three things:

- **Members** (or roles)
- **Workflows** (or ceremonies)
- **Artifacts**

All of these live and breath within a **Sprint**



Scrum – Sprint

A Sprint is the basic unit of time in which the Scrum process takes place. It is usually an allotment of 2 weeks, in which **all** of the Scrum ceremonies take place and repeat themselves.

A Sprint is the beating heart of the Scrum methodology, as it represents the single, briefest, timeslot in which the team is able to output a **working, demoable, vertical, slice of the product**.



Scrum members



Scrum – members

A group of people that work together by applying Scrum methodology are called a **Scrum Team**

A Scrum Team is composed of:

- Product Owner (or PO)
- A Scrum master
- Team members (or developers)

Scrum roles are **not organization roles!**

Examples:

- The team PO **might** be a product manager
- The team Scrum master **might** be a team developer
- A developers might be a software developer, a QA specialist, a data scientist, a designer, etc...



Scrum – members – PO

The Product owner is the main liaison between the team and **all stakeholders! (not just the customer)**

He is responsible for

- Gathering live requirements
- “Protecting” the team during development
- Feeding and ordering the product backlog (more to this later)
- The general outcome of the project
- Represent all stakeholders within the Scrum process (internal and external!)

If the Scrum team is a mini company within a company, the PO is the **CEO!**



Scrum – members – Scrum Master

The Scrum master is the **main facilitator** of the Scrum team!

He is responsible for:

- The general goodness of the Scrum process within the team
- Conducting the Scrum ceremonies (more to this later)
- Coaching other members in being cross functional and self managed
- Removing internal impediments and help the PO in removing external ones
- Helping the PO in producing the best possible artifacts and backlog
- ...and many more!

The Scrum master is tasked with a lot of stuff, and his exact responsibilities vary a lot based on the company, the team and the culture



Scrum – members – Developer

He is not necessarily a software developer; as long as he is a member of the team and he works towards the development of the project, he is a Scrum team developer!

Together, they are responsible for:

- Participate in the Scrum ceremonies to:
 - Refine user stories into tasks
 - Assign story points
 - Plan tasks and stories into a sprint
 - Provide feedback during retrospectives
 - And many more!
- Understanding the items in the product and sprint backlog
- Working the items planned during the sprint planning and strive to complete them



Scrum ceremonies



Scrum – Ceremonies – Daily Scrum

The Daily Scrum (or daily standup, standup meeting) is daily meeting in which **all** team members, individually:

- Discuss what they have achieved the day before
- Discuss what they have planned for the day
- Discuss possible blockers that need to be solved for their work to proceed

In this meeting, the Scrum master will make sure the process moves swiftly (usually less than 15 minutes) and that team members who need further discussion are able to follow up.

Additionally, the Scrum master will take responsibility for informing and organizing the necessary people to unblock other members.



Scrum – Ceremonies – Backlog Refinement

It can either be performed before the start of the sprint or it can be “spread” throughout the sprint as a continuous activity.

Its purpose is to refine the items in the product backlog (user stories) into working items with clear goals, criterias, technical requirements and any other information that might be needed to complete it.

At the same time, the PO is responsible for feeding new items into the product backlog and to sort them by priority.

Once the backlog item is clearly specified, the team developers will assign it a number of **story points** (more to this later).



Scrum – Ceremonies – Backlog Refinement

Example:

Given this high level user story taken from the product backlog:

“As a logged user, I want to be able to access my personal profile, so that I configure my security settings”

The refinement process might raise these discussion points:

“What new API we would need to implement this?” “Do we have a mockup ready for this new section?” “We need a new frontend route for this page”

The team developers would then be responsible to refine these discussion in actual tasks

They are also supposed to interact with the PO – same as if they were consultants interrogating a customer – for any additional detail that is needed for them to create the feature!



Scrum – Ceremonies – Sprint Planning

The sprint planning is the main “contract” between the development team and their PO (and by extension, all stakeholders).

First, a rough idea (**sprint goal**) of the final product for the sprint is agreed between the team members; this is normally a **thin, vertical, slice of product**.

In this meeting, the team developers will go over the **ordered** list of items from the product backlog and decide how many of them they will be able to complete (or “fit”) within the next sprint, with the intention of implementing the sprint goal. This decision is mainly based on the amount of **total story points** they can normally “burn” throughout a sprint.



Scrum – Ceremonies – Sprint Review

The sprint review is the moment of truth for the achievements of the sprint. Conducted at the very end of the sprint, the team illustrates the results of their efforts to the PO (but also other stakeholders are normally invited to participate).

This moment is where feedback is provided, to either approve the direction or gather requirements to **steer the development**.

There are many formulas to sprint reviews, such as presentations, live demos, sprint bazaars.



Scrum – Ceremonies – Retrospective

A retrospective is usually conducted at the absolute end of the sprint (usually starting the weekend!).

In the same way the scrum team iterates to improve on their product, they also **iterates to improve on their own scrum team and process.**

There are many ways to conduct a retrospective, but generally speaking, the Scrum master is tasked to collect feedback from the rest of team to understand:

- What went well
- What went wrong
- Define action items to improve and follow up on them

The scrum master is the responsible for coaching the other team members, so he is the one who will ping you to check if you did what you were asked to during the retro!



Scrum artifacts



Scrum – Artifacts – Product Backlog

It is **ordered list of work items**, usually in the form of **user stories, technical tasks, investigations** and anything in between!

The product owner is the main responsible for maintaining and prioritizing the product backlog by considering their business value, time constraints, complexity, size and other factors.

The top items will be the first to be looked at during the **sprint refinement** and the ones more likely to be assigned story points, as they are also the main items for the next sprint.



Scrum – Artifacts – User story

A user story is an **informal, written description of an interaction between a user and the system.**

There are various formats for user stories, but the most common one is the simple, sentence based,

“As <x>, I want <y>, so that <z>” (Gherkin)

Usually, though, sprint refinements will need to expand on this information, requiring more formal and/or expanded descriptions.



Scrum – Artifacts – User story

A very common, more detailed, structure is **gherkin**.

Gherkin further expands on user stories definitions with the following format:

Feature: brief title

Scenario: brief description

Given <x>

And <y>

When <k>

Then <z>

For example, using the requirement from before

Feature: User profile page

Scenario: Opening a personal profile page

Given that I am a user

And I am logged in into the system

When I click my profile icon

Then I am redirected to my profile page



Scrum – Artifacts – Sprint Backlog

After the Sprint planning, the contracted items are then moved to the sprint backlog, where they enter the development process (usually as items of a development board).

Team developers will then assign the tasks from this backlog to themselves **independently** and move them throughout their SDLC.



Scrum – Artifacts – Sprint board

Is a project tracking device that allows every team member to see a snapshot of what is being worked and what its status is, as well as allowing them to move work items forward.

There are a number of board technologies and standards; generally scrum boards will require items to flow from left to right, starting from the sprint backlog column, up to a “done” column.

The **definition of “done” (DoD)** is an important aspect of the scrum team, that is carefully established and updated. Each team and organization will have their own definition of done, depending on the actual release process, work that needs doing from other teams, etc...



Pitfalls and downsides



Scrum – Pitfalls

- Story points used as a measure of time
- PO or other managers assigning work and micromanaging
- Misusing ceremonies and doing them because “we have to”
- Trying to force scrum; not all project and teams benefit from it!



Scrum – Downsides

- Overall time to completion can be large and difficult to track
- It requires commitment and knowledge from all members of the team
- Doing little vertical slices forces team to rework items
- Encouraging requirements changes from the customer will force team to rework items



Measuring scrum



Scrum – Measuring Scrum

You cannot improve what you can't measure. But how do you measure your scrum process?

There are many ways (usually called scrum metrics), some examples:

- Velocity
- Lead time
- Burndown chart
- And many others

The scrum master is responsible for measuring and translating the measurement in action items (especially during the retro!)



Philosophy



Scrum – Philosophy

There is **ONE** main point of Scrum: the paradigm shift from

The development team obeys a management process

To

The development team is an independent, trustworthy, being that solves problems



Scrum – Philosophy

In a healthy Scrum organization, the team acts almost as an external company that is capable of implementing solutions. They don't blindly obey tasks "from above".

The team members (you) are responsible. With pros and cons (lots of meetings? perhaps...)



Scrum – Philosophy

Failing to understand and apply this concept is the main reason why:

- Agile is often hated
- Agile is often “forced”
- Agile is often perceived as “lots of extra useless meetings”
- So on...



Scrum – Philosophy

This also implies that not all companies can work with Scrum, and that not all teams (and individuals) are a good fit for Scrum.

A good company has the responsibility to decide what's best for the teams.